

Introduction to Computer Architecture

Chapter 2

Instructions: Language of the Computer - 3

Hyungmin Cho

Department of Computer Science and Engineering
Sungkyunkwan University

Procedure Calling

Caller

```
int main() {  
    ...  
    ...  
    arg2 = val1 * 2;  
    result = compute(arg1, arg2);  
    ...  
    ...  
}
```

arguments
↓ ↓
a0 a1
register

Callee

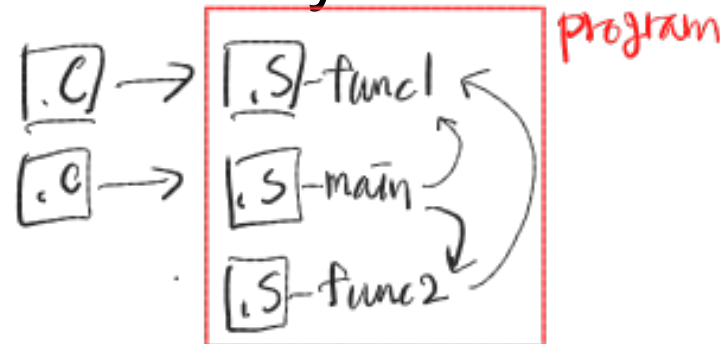
```
int compute(int p1, int p2) {  
    int temp1;  
    int temp2;  
    ...  
    temp1 = p1 * p2;  
    temp2 = temp1 + 10;  
    ...  
    return temp2;  
}
```

parameters
↑ ↑
a0 a1

1. How to pass parameters (arg1 and arg2) to the **callee** (function)
2. How to reserve memory space for the callee's **local variables**
3. How to transfer the result (return value) back to the **caller**
4. How to return to the caller

Procedure? Function?

- Are they the same? ... Not exactly.
- Procedure is a group of instructions that are invoked to perform a designated task.
- There is no concept of “Function” in assembly language. *Function은 high-level language의 개념*
- We use a special rule to support “C”-style function in assembly
→ calling convention



RISC-V Register Usage

Calling Convention

16 32 registers
4 80000 16 32

Register Number	Name	Usage
x0	zero	Constant 0 (hardwired)
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5-x7, x28-x31	t0 - t6	Temporaries
x8	s0 / fp	Frame pointer
x9, x18-x27	s1 - s11	Saved registers
x10-x11	a0 - a1	Function arguments / results
x12-x17	a2 - a7	Function arguments

t

s

a

126

5

12

8

Argument Passing

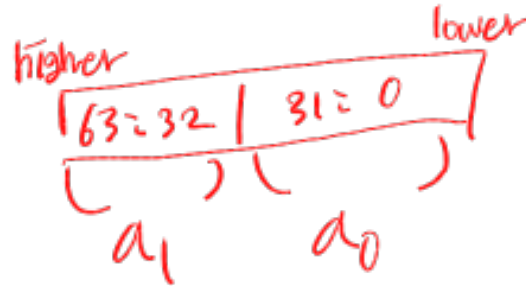
- Set register $x10 - x17$ with the arguments in order **before calling**
 $(a0) \quad (a7)$
the function.
- If the function takes more than 8 arguments, the rest of the arguments are passed to the callee using memory.

Return Value

- Use register x10 - x11
return 할 때 쓰일
a₀ a₁

- ❖ x10 [31:0]
- ❖ x11 [63:32]

*64bit의
값을 넘겨줌*



*python과 같이
컴퓨터를 위한 값으로
return*

- If the return value is larger than 64-bit, the return value is passed through memory.

Procedure Call Instructions

■ Procedure call: jump-and-link instruction

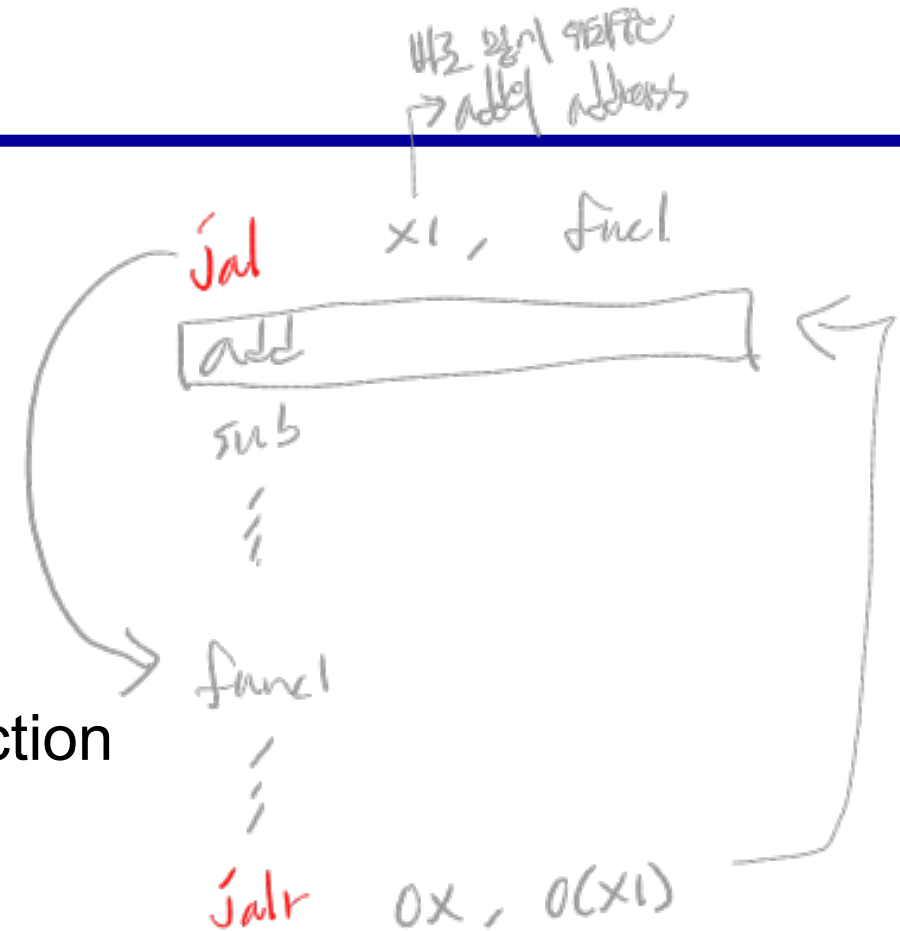
jal x1, Procedure_Label

- ❖ Jumps to the target address
- ❖ Save the address of the next instruction in x1
- ❖ x1's register name is ra (return address)
procedure의 바로 뒤로 return하는 지

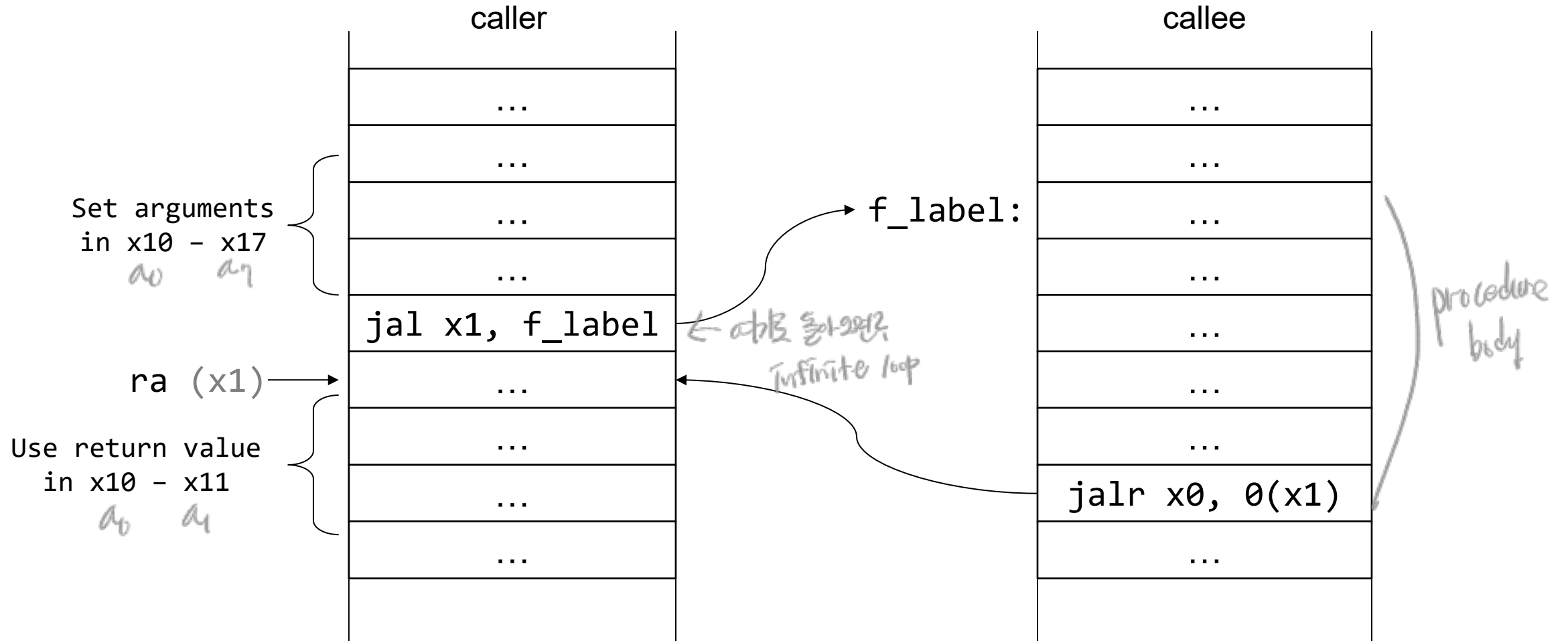
■ Procedure return: jump-and-link register instruction

jalr x0, Offset(x2)

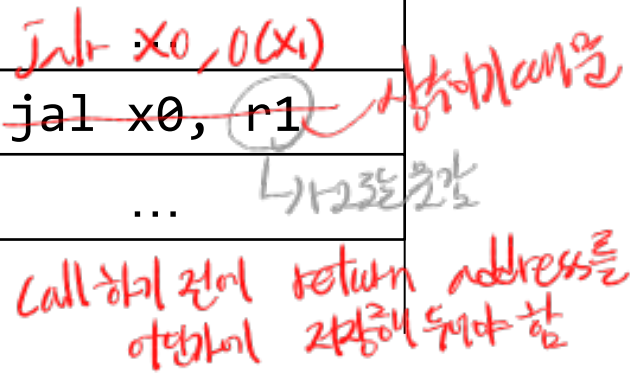
- ❖ Jump to offset + address in x2
- ❖ Address of the next instruction → x0
- ❖ Can also be used for computed jumps
 - e.g., for case/switch statements



RISC-V Procedure Call



functions을 쓰는 이유는
다들 공부해서 아는데 개인용이
아니



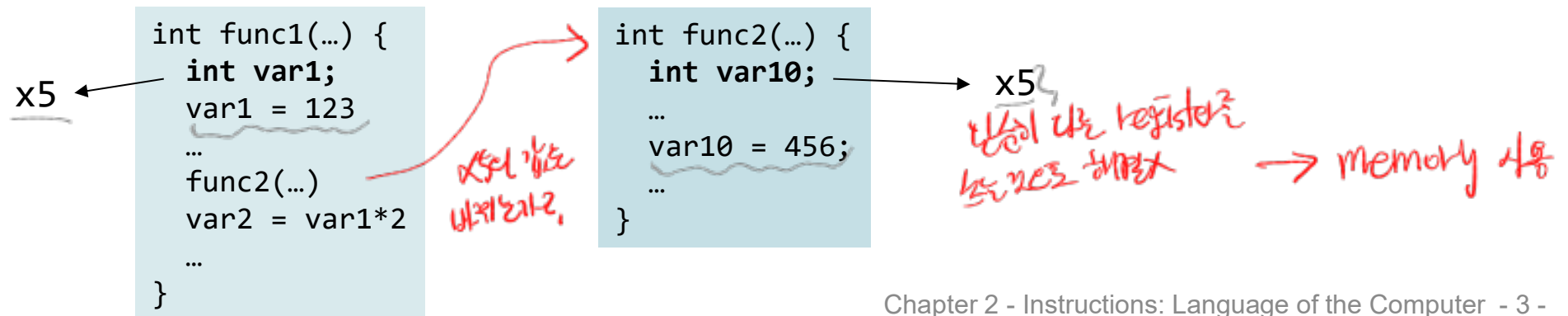
Chapter 2 - Instructions: Language of the Computer - 3 -

Memory Space for Procedure (Function)

- A procedure needs its own memory space for the local variables.

```
int func1(...) {  
    int var1;  
    int arr2[2];  
    ...  
}
```

- ❖ If all local variables can fit in the registers, a procedure may not need to use memory space (compiler optimization)
- ❖ Even all local variables can fit in the registers, the values may need to be saved in the memory to call another function.



Memory Space for Procedure (Function)

- How to reserve space for each procedure?
 - ❖ A fixed location for each procedure **is not an ideal solution**
 - We don't know which procedures will be called in advance
 - Can't support Recursion!

Fixed address?

```
int func1(...) {  
    int var1;  
    int arr2[2];  
    ...  
}
```

→ 0x1000
→ 0x1004

static fixed
11 x 5, 0x1000
1W x 6, 0(x5)

```
int func2(...) {  
    int arr20[100];  
    int var10;  
    ...  
}
```

→ 0x2000
→ 0x2190

fixed address? X
Function! recursion 일 때,
계속 같은 곳에 재호출이 됨

Memory space

Space for func1

Space for func2

Space for func3

⋮

Space for funcN-1

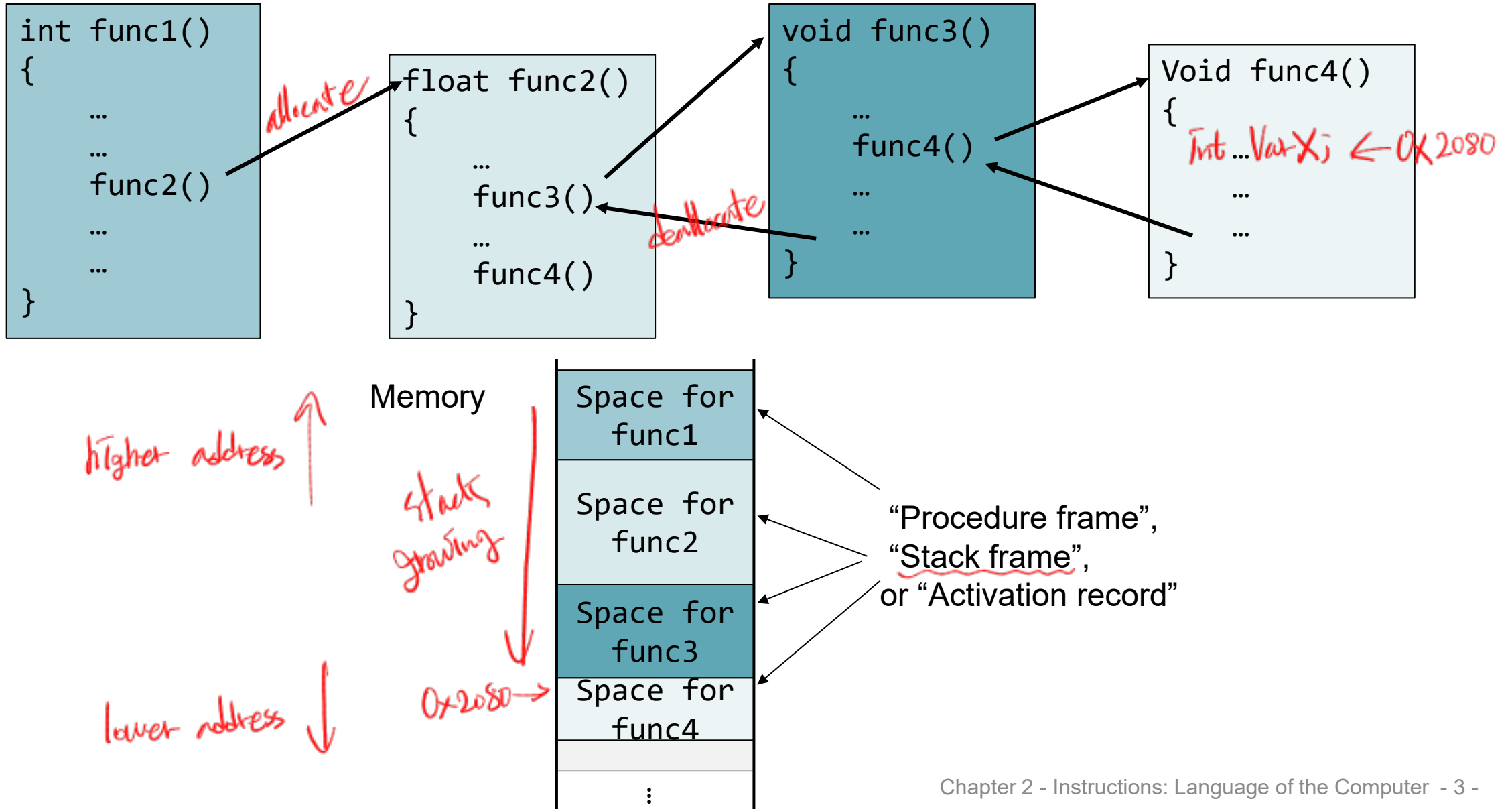
Space for funcN

■ Solution?

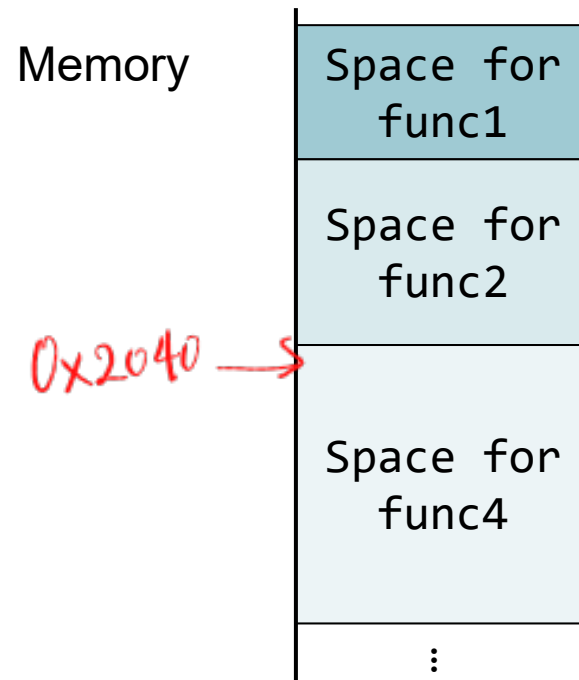
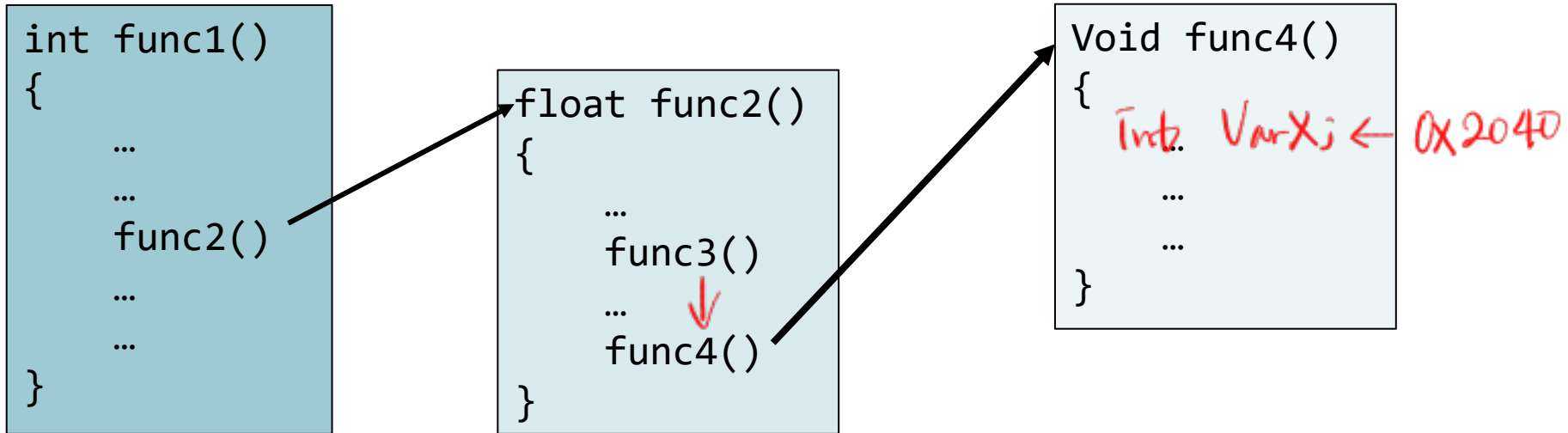
- ❖ “Allocate” the space when the function is called / “Deallocate” when returns
- ❖ Use relative addressing within the allocated space.

사전에 할당 받기

Stack



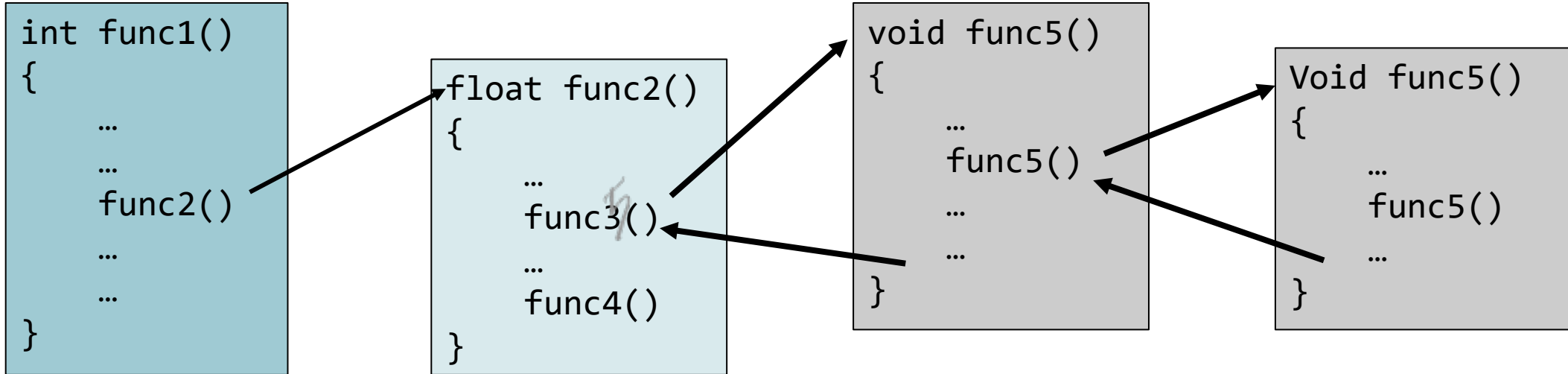
Stack



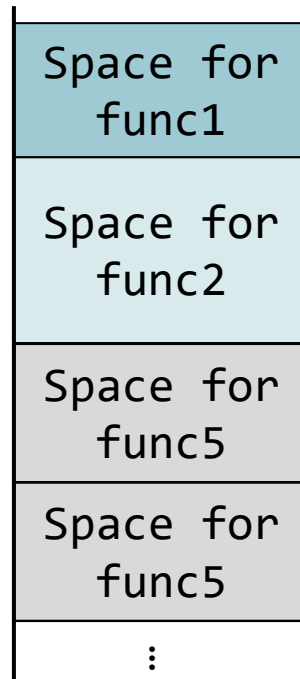
li x5, 0x2040 → func4의
0x2080의 값을 가져와서
리시프트해서
↓
relative addressing

Stack

four five



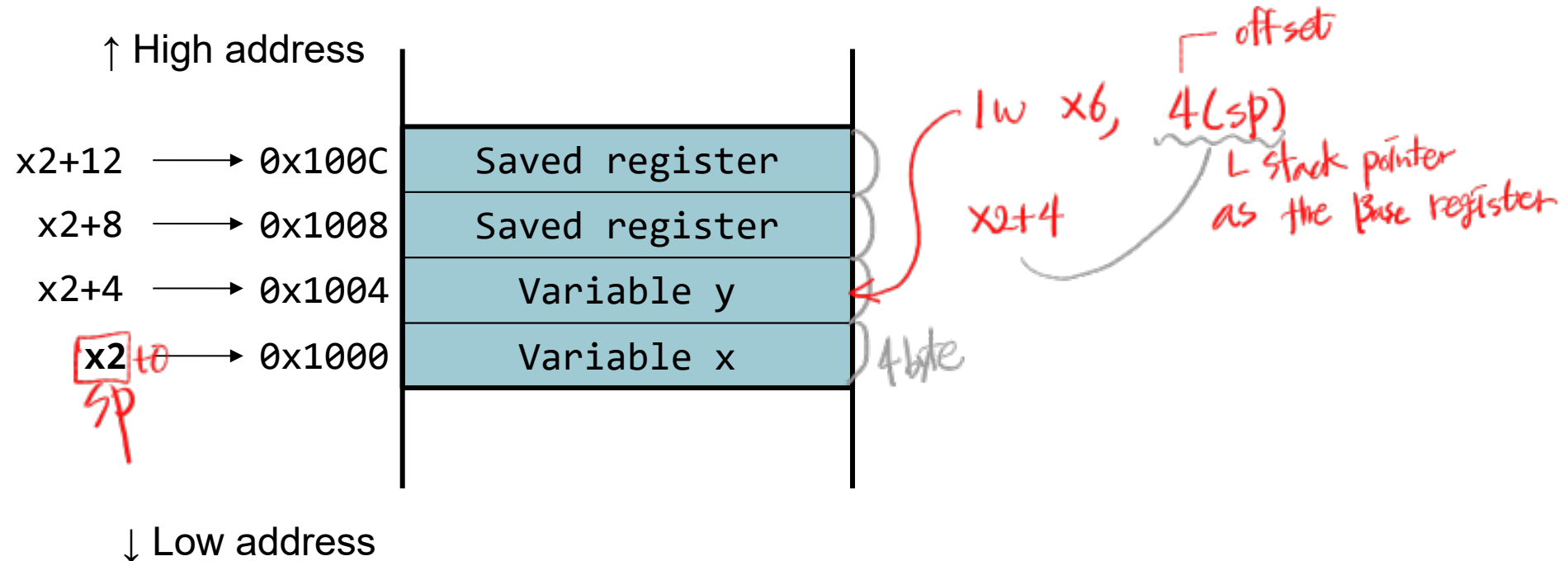
Memory



Managing & Accessing Stack (1)

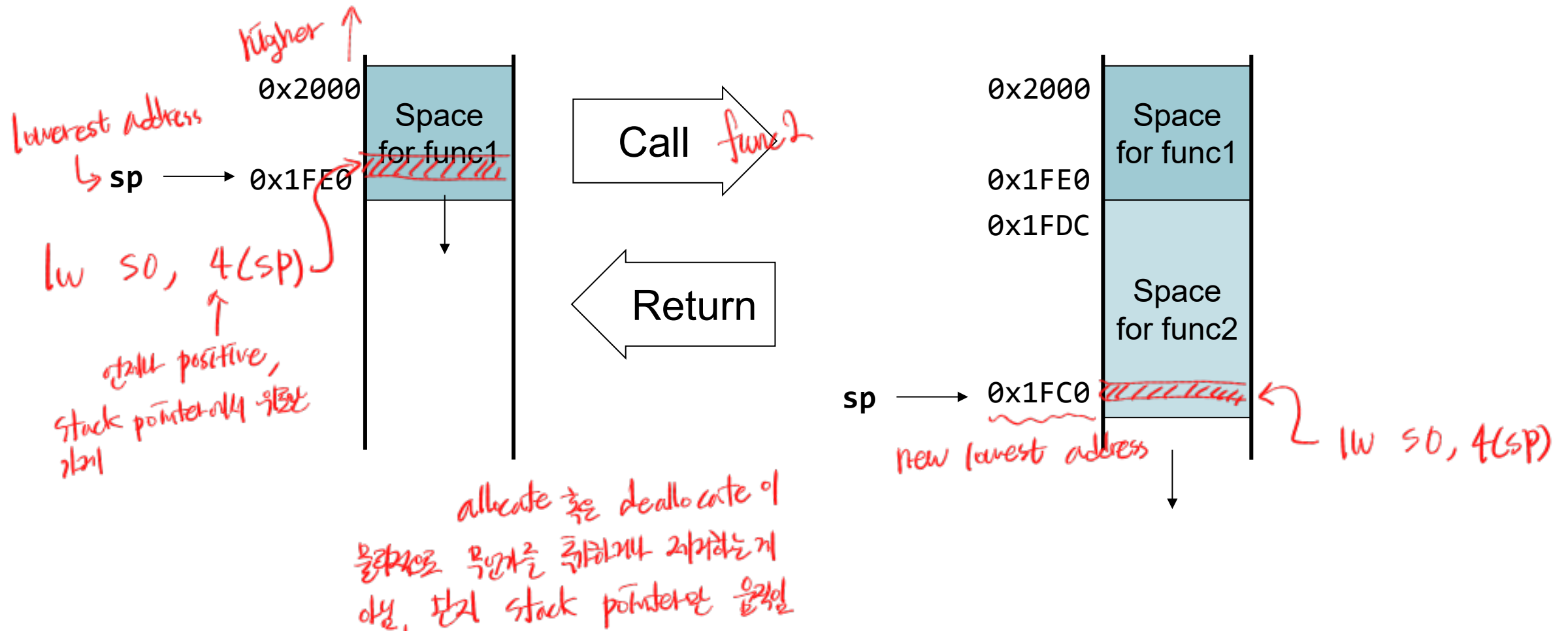
relative addressing

- A stack frame can be placed at any location in memory
 - ↳ Relative addressing is required
- Stack pointer register x2 (sp) points to the lowest address in the current stack frame



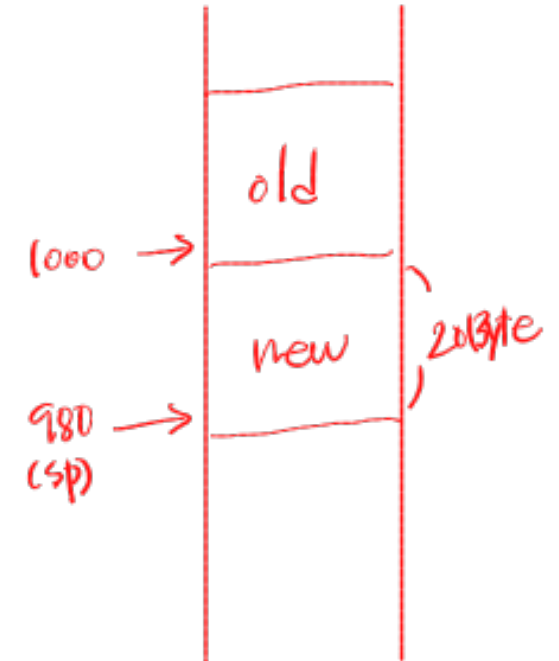
Managing & Accessing Stack (2)

- Stack grows from higher address to lower address

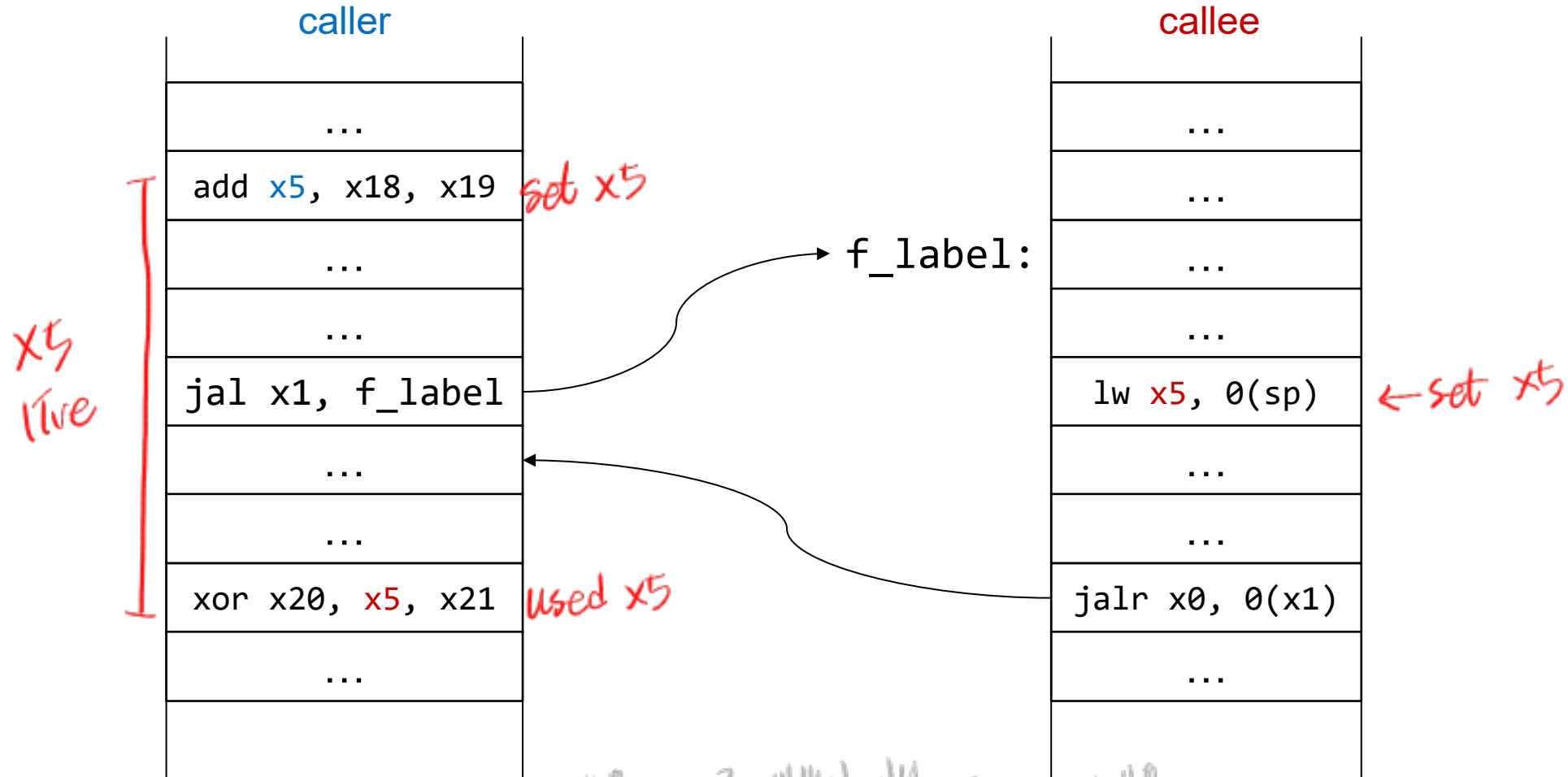


Managing & Accessing Stack (3)

- Each procedure needs to know how much ^{stack frame size} stack space it needs
 - ❖ High-level language: the compiler automatically computes the required space
 - ❖ Assembly language: the programmer need to determine the required size
- Suppose a function requires 20 bytes of space (e.g., 5 x 4-byte values)...
 - ❖ When the function starts, **decrement** `sp` by 20
 - `addi sp, sp, -20`
 - ❖ To access a stack value, use `sp` as the base register
 - `sw x18, 4(sp)`
 - ❖ When the function returns, **increment** `sp` by 20
 - `addi sp, sp, 20`



Saving Registers



register collision을 막기 위해 odd #인 사용 → 짝수 번째가 아님 ← even #인 사용

Register Saving Convention

callee caller 중이 누가 save해야 하는지
register type에 따라
save 하는

comment flow를 따라가

Register Number	Name	Usage
x0	zero	Constant 0 (hardwired)
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5-x7, x28-x31	t0 - t6	Temporaries
x8	s0 / fp	Frame pointer
x9, x18-x27	s1 - s11	Saved registers
x10-x11	a0 - a1	Function arguments / results
x12-x17	a2 - a7	Function arguments

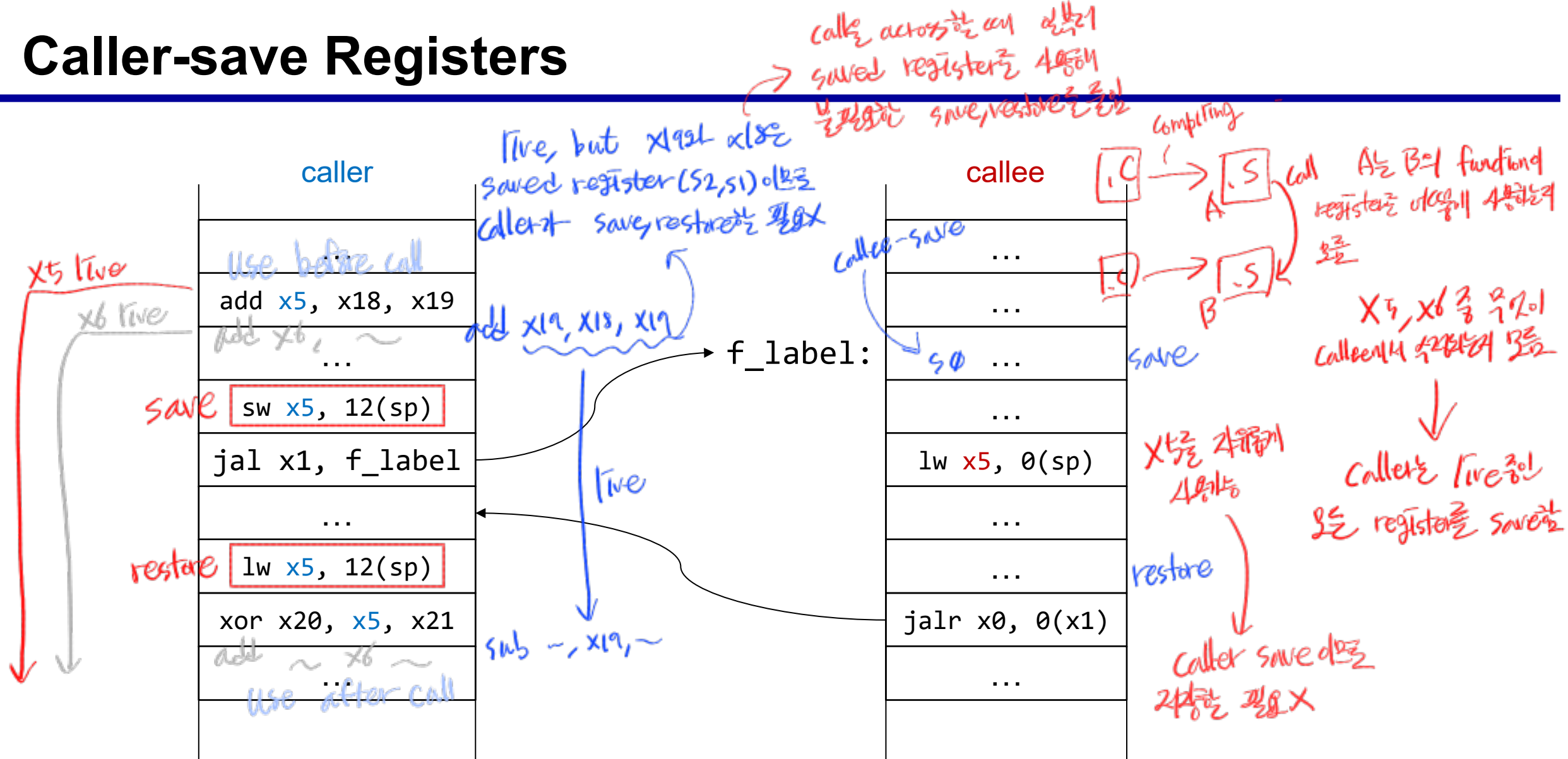
Caller-save registers

Callee-save registers

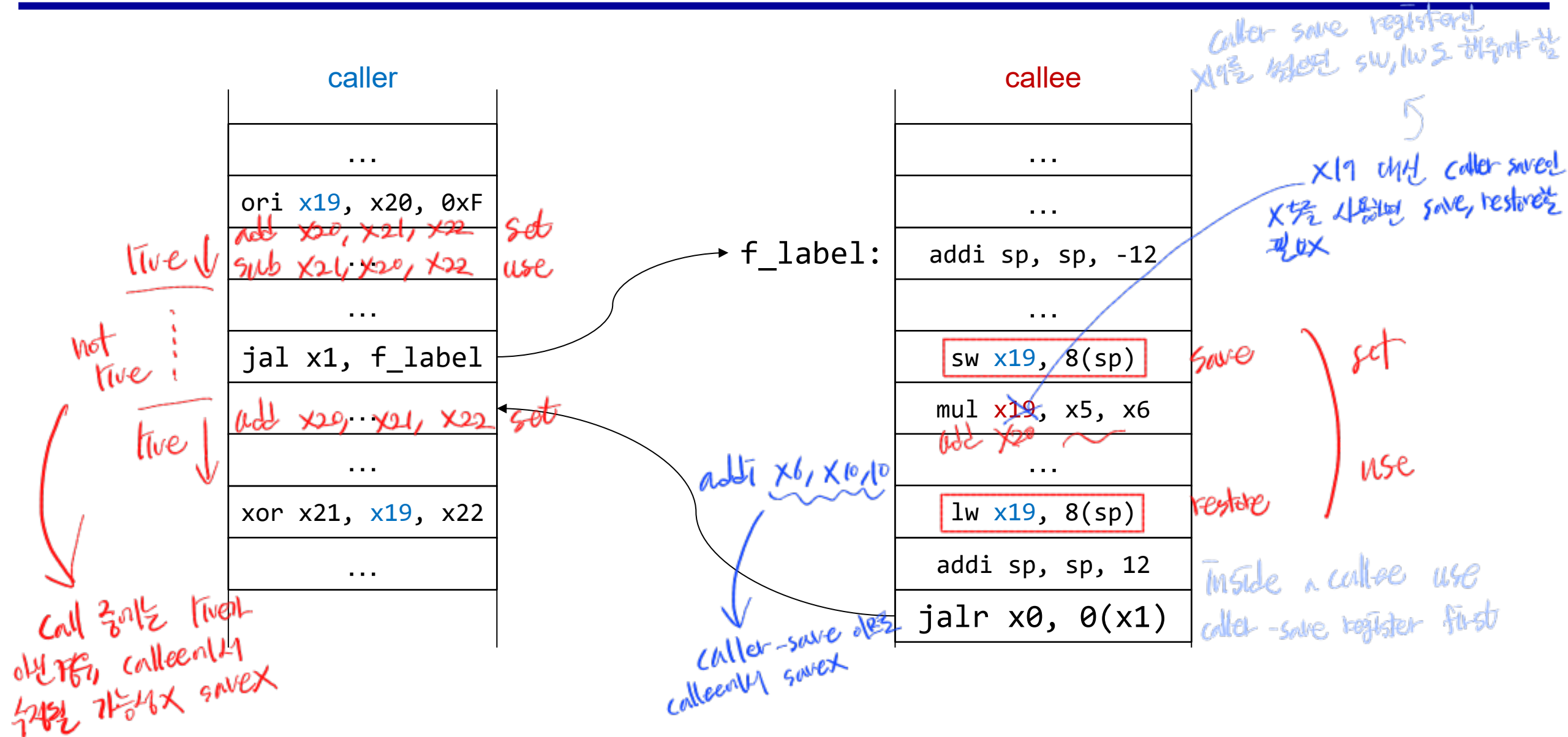
If the caller wants to **use value** in these **registers** *after* calling a function,
→ caller need to store these registers in its stack frame before calling the function

If the callee wants to **use these registers**,
→ callee need to store these registers in its stack frame and restore those before the return

Caller-save Registers



Callee-save Registers



Avoiding Unnecessary Register Save & Restore

- If a value is used for a short period, use the “Temporary” registers
(caller-save)

instead of the “Saved” registers
(callee-save)

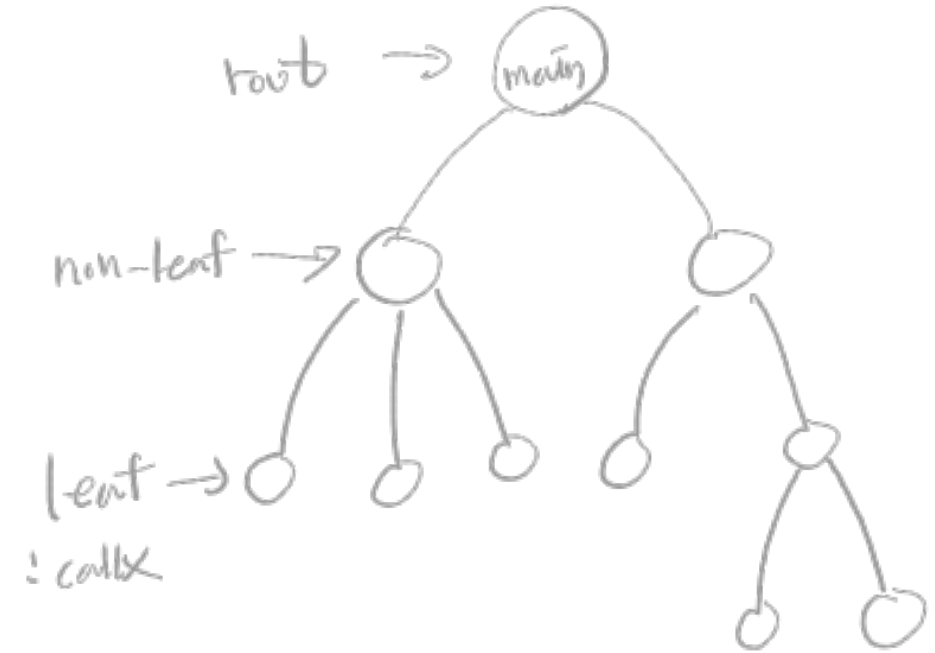
Caller에서 이걸 할지 물어볼지
Caller는 알 수 없음, (callee도 모른다)

- Use “Saved” registers if a value is created before a function call and used after the function call.
- Use “Argument” and “Results” registers as temporary registers if possible

Leaf Procedure Example

■ C code:

```
int leaf_example (int g, h, i, j)
{
    int f;
    f = (g + h) - (i + j);
    return f;
}
```



- ❖ Arguments g, ..., j in x10, x11 ..., x13 (a0, a1, ..., a3)
- ❖ f in x20 (s4)
 - hence, need to save the caller's x20 on stack
- ❖ Temporaries x5 (t0), x6 (t1)
- ❖ Return value in x10 (a0)

⇒ 4 byte

Leaf Procedure Example

```
int leaf_example (int g, h, i, j)
{
    int f;
    f = (g + h) - (i + j);
    return f;
}
```

$x_9, x_{18}, x_{19}, x_{20}$
 s_1, s_2, s_3, s_4

leaf_example:

prologue

```
(addi sp, sp, -4
sw    x20, 0(sp)
add    x5, x10, x11
add    x6, x12, x13
sub    x20, x5, x6
addi   x10, x20, 0
lw     x20, 0(sp)
addi   sp, sp, 4
jalr   x0, 0(x1)
```

epilogue

unnecessary moving
x20 of x20 & x20

Allocate stack space

Save x20 on stack

s_4

Procedure body

Set return value val

Restore x20

De-allocate stack space

Return

Leaf Procedure Example (2)

```
int leaf_example (int g, h, i, j)
{
    int f;
    f = (g + h) - (i + j);
    return f;
}
```

leaf_example:

```
add    x5, x10g, x11h
add    x6, x12i, x13j
sub    x10no, x5, x6
jalr   x0, 0(x1)
```

no callee saved
register used



no save, restore
= no allocate, deallocate
= simple

Procedure Call Example

caller side

```
int a, b, c, d, e;  
...  
d = b*c;  
e = leaf_example(10, a, b+c, d);  
e = e + d;
```

- ❖ a at sp + 0
- ❖ b at sp + 4
- ❖ c at sp + 8
- ❖ d at sp + 12
- ❖ e at sp + 16

a0-a3

```
lw    t0, 4(sp)  
lw    t1, 8(sp)  
mul   s4, x5, x6  
addi  a0, zero, 10  
lw    a1, 0(sp)  
add   a2, t0, t1  
add   a3, s4, zero  
jal   x1, leaf_example  
add   t0, a0, s4  
sw    t0, 16(sp)
```

b

c

d in s4 = b*c

Parameter 0 : 10

Parameter 1 : a

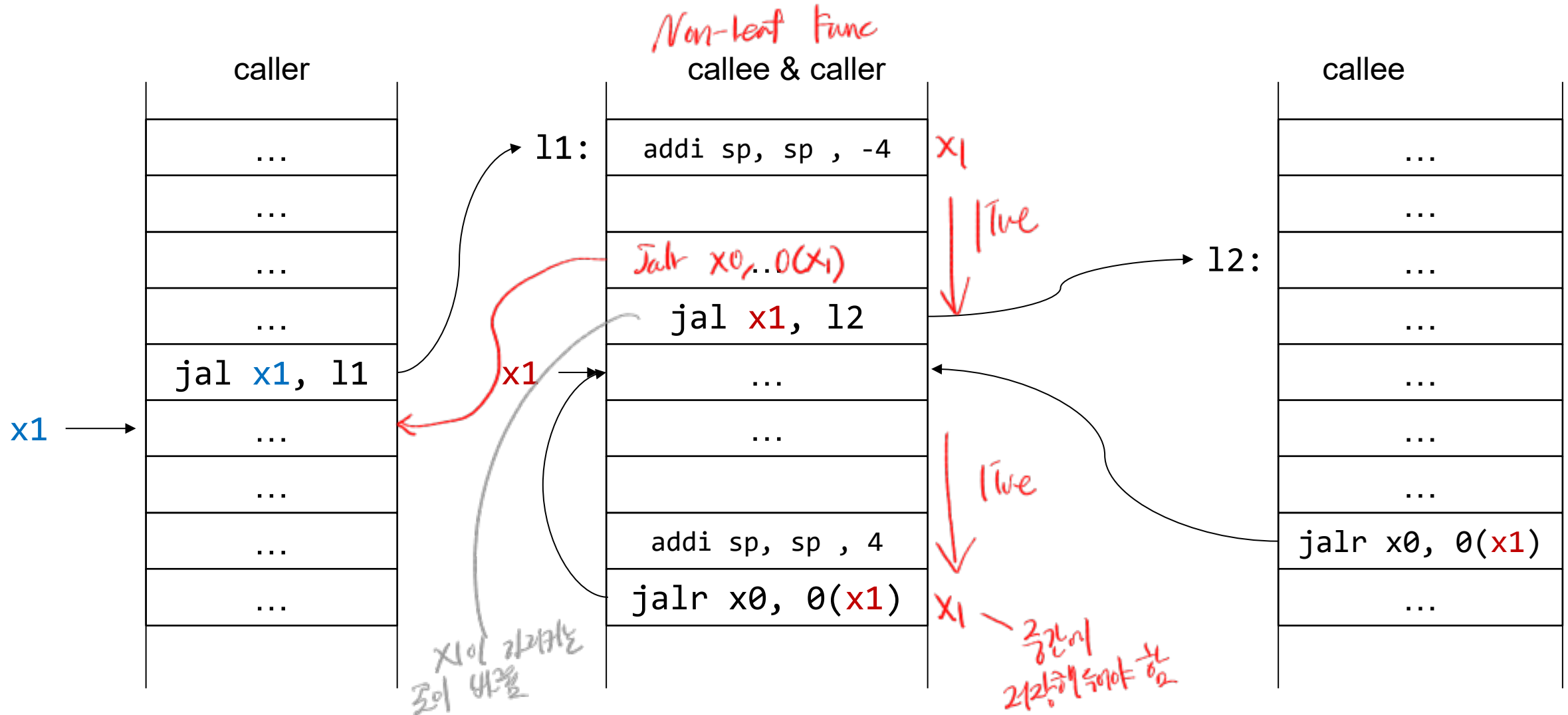
Parameter 2 : b+c

Parameter 3 : d(s4)

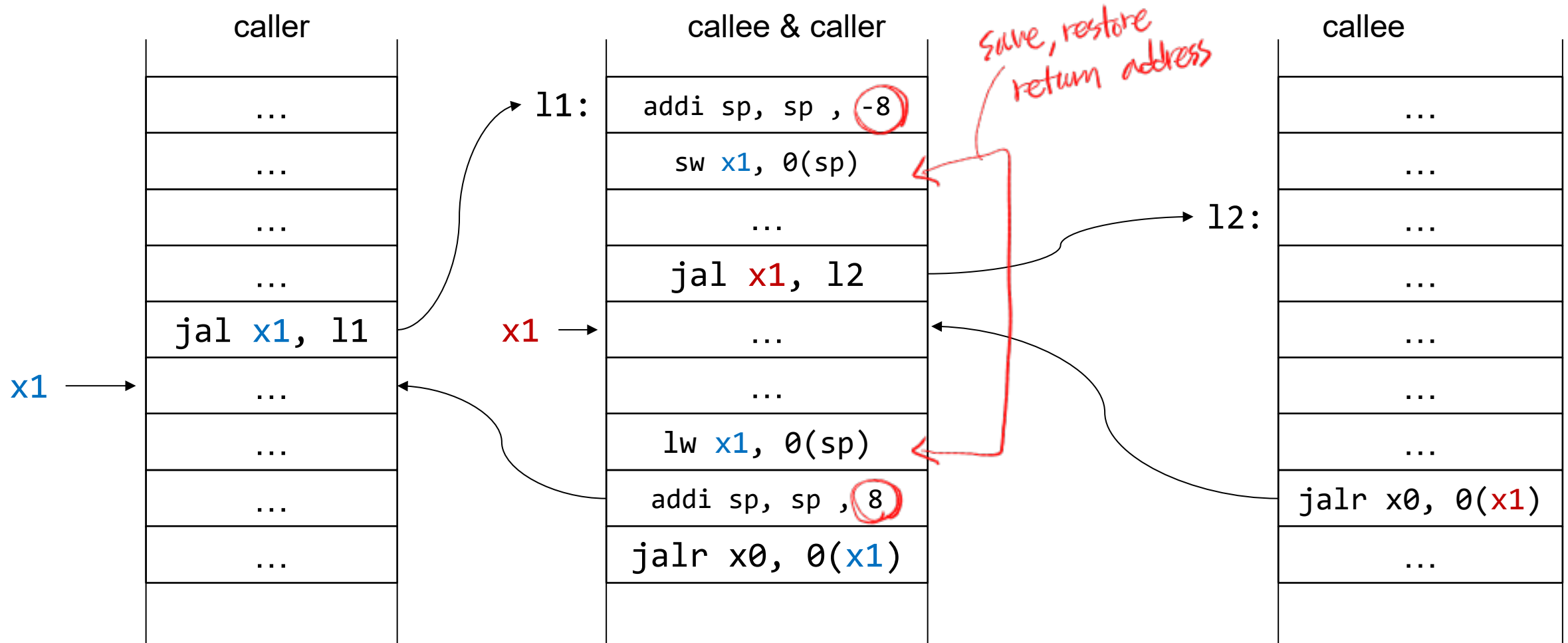
call

d is still in s4

Procedure Calls – Non-Leaf



Procedure Calls – Non-Leaf



Non-Leaf Procedure Example

■ C code:

```
int fact factorial (int n)
{
    if (n < 1) return 1;
    else return n * fact(n - 1);
}
```

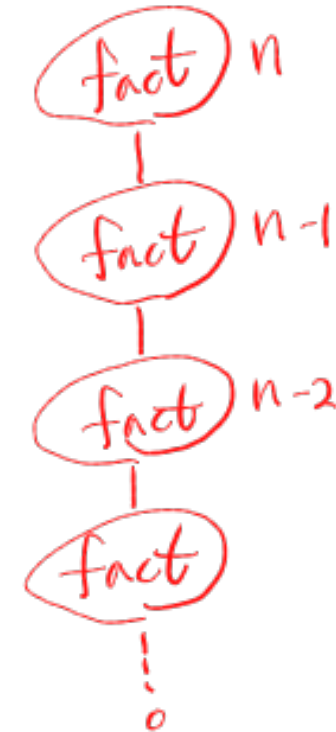
❖ Argument n in x10 *do caller-save*

❖ Result in x10 *do*

❖ Need to store return address (x1) in stack

❖ Need to store argument (x10) in stack

➢ n is used again after calling fact(n-1)



*2, 4 Byte
8 Byte*

Non-Leaf Procedure Example

```
int fact (int n)
{
    if (n < 1) return 1;
    else return n * fact(n - 1);
}
```

fact:

```
addi sp, sp, -8
sw x1, 4(sp)
sw x10, 0(sp)
```

allocate stack for two 4-byte items
save return address
save argument (n) saved

```
addi x5, x0, 1
bge x10, x5, L1
```

test if $n < 1$

```
addi x10, x0, 1
addi sp, sp, 8
jalr x0, 0(x1)
```

if $n < 1$, result is 1
de-allocate stack space
return

```
L1: addi x10, x10, -1
jal x1, fact
```

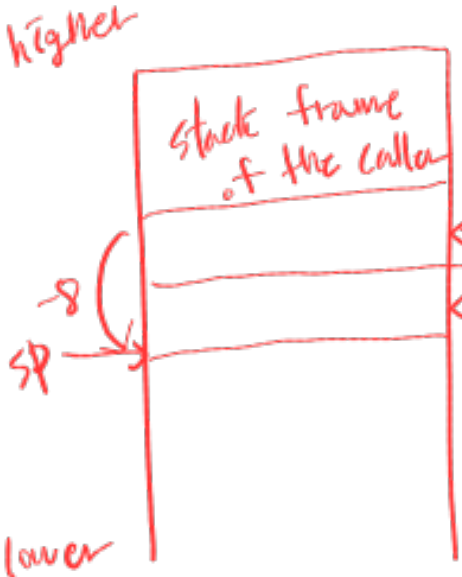
if $n \geq 1$ set $n-1$ as argument
recursive call

```
lw x5, 0(sp)
mul x10, x5, x10
```

restore original n
multiply to get result

```
lw x1, 4(sp)
addi sp, sp, 8
jalr x0, 0(x1)
```

restore return address
de-allocate stack space
return



$n < 1$

$n \geq 1$

else:

return

$x10 \rightarrow \text{fact}(n-1)$

to $n \times \text{fact}(n-1)$

No restore operation

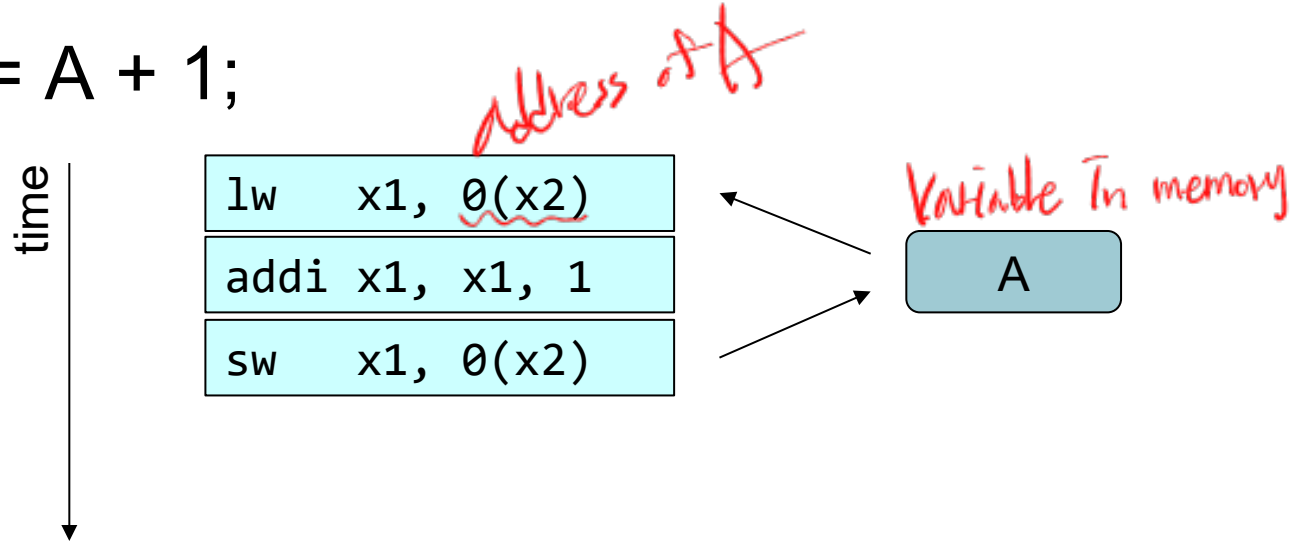
여기서 $x10$ 은 쓸 필요가 없음

$x1$ 이 반환값을 담는 역할을 함

각각 명령어 주변 기능

Synchronization

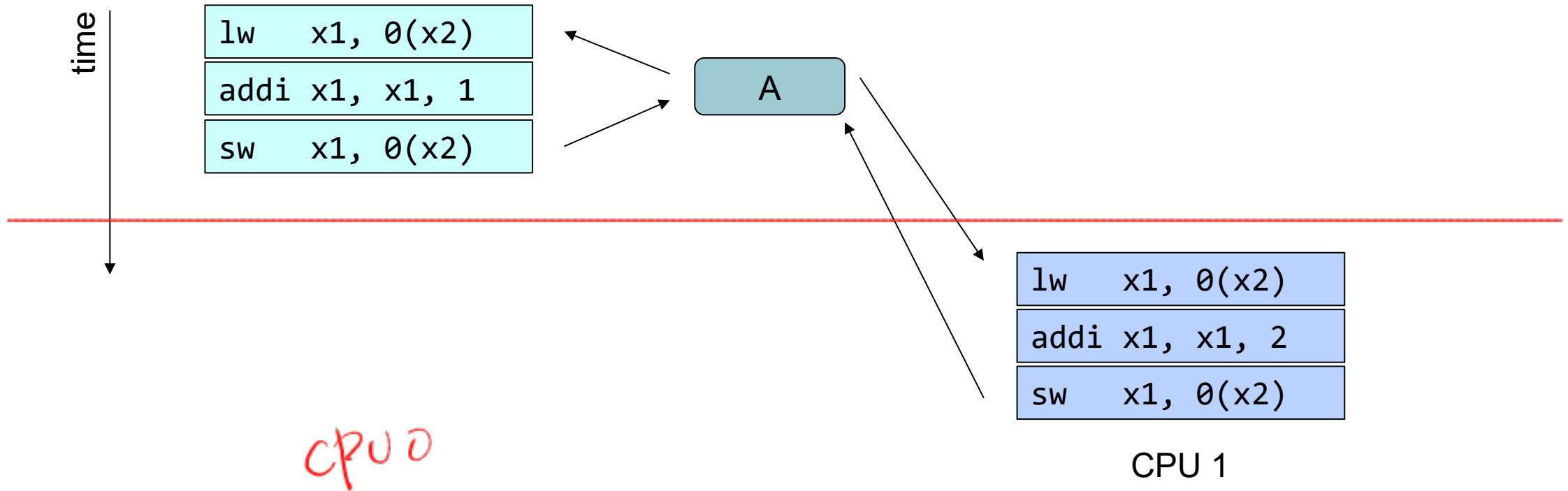
- $A = A + 1;$



Synchronization

■ $A = A + 1;$

■ $A = A + 2;$



CPU 0

CPU 1

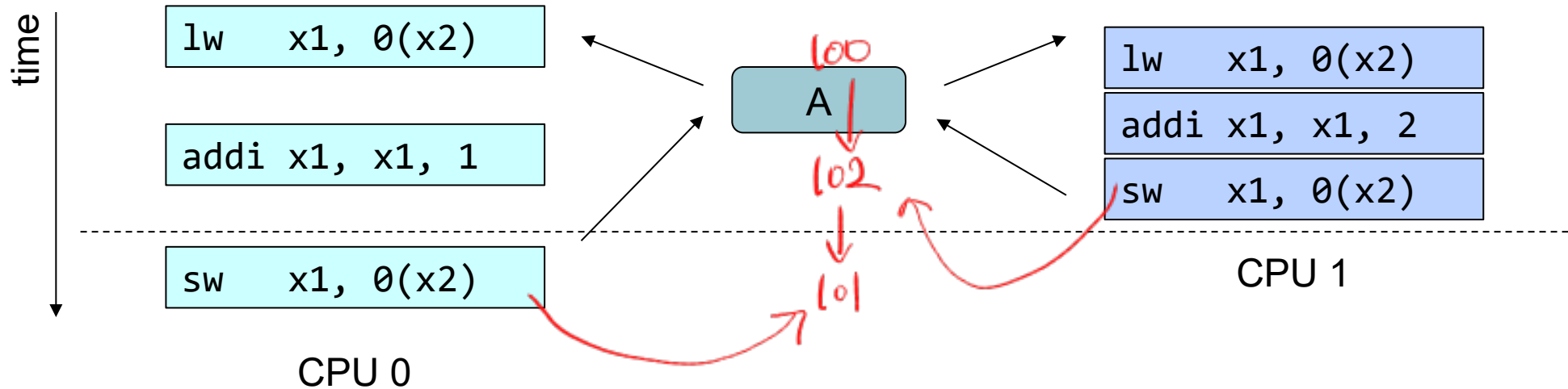
순서대로 이루어진다면

$A = A + 3$

Synchronization: Race Condition

■ $A = A + 1;$

■ $A = A + 2;$



$A = A + 1$??

Data “race”

Synchronization: Atomic Operation (1)

■ Special memory instructions (RISC-V “A” extension)

lw x1, 0(x2)	→	lr.w x1, x (x2)
addi x1, x1, 1		addi x1, x1, 1
sw x1, 0(x2)	→	sc.w x1, x (x2)

❖ lr.w : load word **reserved**

➢ Load from address in x2 to x1 (no offset)

❖ sc.w : store word **conditional**

➢ Store from x1 to address in x2 (no offset)

■ sc.w writes to the memory only if the target location wasn't modified after lr.w

❖ → x1 is set to 0

■ If the memory location was changed by someone else, store does not happen

❖ → x1 is set to a non-zero value

Zero → succeeded

offset x
same address
race condition
sc.w
*2개나 다른 processor가
동시에 접근*

Synchronization: Atomic Operation (2)

■ $A = A + 1;$

```
lr.w  x1, (x2)
```

```
addi x1, x1, 1
```

```
sc.w  x1, (x2)
```

CPU 0

*failed x1 ≠ 0
A = A + 1*

- Write does not happen
- x1 becomes non-zero
- Your program may choose to retry the task if x1 is non-zero.

■ $A = A + 2;$

```
lr.w  x1, (x2)
```

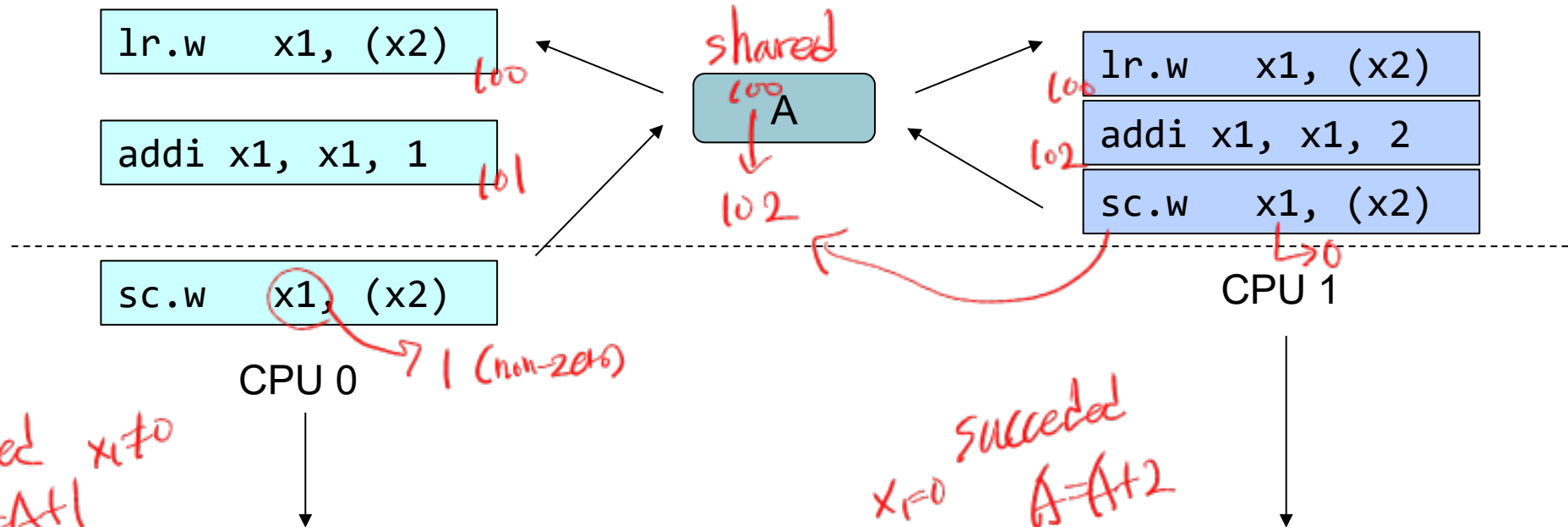
```
addi x1, x1, 2
```

```
sc.w  x1, (x2)
```

CPU 1

*succeeded
x1 = 0
A = A + 2*

- Write happens
- x1 becomes 0
- Your program can proceed to the next task if x1 is 0.



Synchronization

- In high-level languages, you can use these basic synchronization methods through library functions
- In Linux, POSIX pthread library
 - ❖ `pthread_mutex_lock()`
 - ❖ `pthread_mutex_unlock()`